

Welcome to 151B: Deep Learning

Instructor: Annapurna Vadaparty

TA: Ben Bao

Tutors: Alexander Radulescu, Trevor Duong

Upcoming deadlines

- Signed up for Thursday oral assessment?
- Signed up for Thurs/Fri CBTF/TTC exam?
- Submissions for A1 part 1 going well? (Part 2 deadline moved to next week)

Oral Assessment Practice

- Questions in the programming portion–hopefully helpful to think through along with sample oral assessment questions to be released soon
- Sample Oral Assessment will be posted for you to work together on during the end of class

Tricks of the Trade

Changes to input handling:

- Stochastic gradient descent versus batch learning
- Shuffling the examples
- Perform PCA of the inputs

Changes to architecture/model properties:

- Change the sigmoid
- Initialize the weights
- Momentum
- Adaptive learning rates

Tricks of the Trade–these will now be part of your competitive advantage

≡ Competitive advantage

Article [Talk](#)

From Wikipedia, the free encyclopedia

Not to be confused with [Absolute advantage](#) or [Comparative advantage](#).

In business, a **competitive advantage** is an attribute that allows an organization to outperform its [competitors](#).

A competitive advantage may include access to [natural resources](#), such as high-grade ores or a low-cost power source, highly skilled labor, geographic location, high entry barriers, and access to new technology and to proprietary information.

Tricks of the Trade

Changes to input handling:

- **Stochastic gradient descent versus batch learning**
- Shuffling the examples
- Perform PCA of the inputs

Changes to architecture/model properties:

- Change the sigmoid
- Initialize the weights
- Momentum
- Adaptive learning rates

Batch Gradient Descent

- This is a learning method where:
 - 1. Get one example
 - 2. Compute the gradient
 - 3. Add it to a running average of the gradients
 - 4. If you have seen all of the examples, now change the weights
 - 5. Go to 1.
- This is following the true gradient for the error function
- You only update the weights once per epoch – one pass through the entire training set.

Stochastic Gradient Descent (SGD)

- This is an online (as-the-examples-come-in) learning method:
 - 1. Get one example
 - 2. Compute the gradient
 - 3. Change the weights
 - 4. If you have seen all of the examples, reshuffle them
 - 5. Go to 1.
- Why is this stochastic?

Stochastic Gradient Descent (SGD)

- This is an online learning method:
 - 1. Get one example
 - 2. Compute the gradient
 - 3. Change the weights
 - 4. If you have seen all of the examples, reshuffle them
 - 5. Go to 1.
- It is *stochastic* because after you have gone through all of the examples (called one epoch), the examples are re-shuffled – so the actual gradients you follow are in random order
- And the gradients are computed for each example – so sometimes they can get around local minima, because the model is not following the true (total) gradient. (Why?)

Why Stochastic gradient descent?

- It tends to learn faster due to redundancy in the training set – for example, MNIST
- You can learn a lot about the problem well before you have seen all of the examples
- It tends to get better solutions, and better generalization (**why?**)

Why Batch learning?

- Convergence conditions are better understood– in general, there's a fair amount of theory.
- Better optimization techniques exist, and there are 2nd order techniques that use the 2nd derivative as well
- An empirical point–this aggressive optimization can lead to large weights – with poorer generalization.

Tricks of the Trade

Changes to input handling:

- Stochastic gradient descent versus batch learning
- **Shuffling the examples**
- Perform PCA of the inputs

Changes to architecture/model properties:

- Change the sigmoid
- Initialize the weights
- Momentum
- Adaptive learning rates

Shuffle the examples (SGD)

1. In particular, shuffling means that we will see patterns from different classes.
2. It is important that a mini batch have most of the categories in it, so the model learns to discriminate between the classes (**otherwise...?**)
3. A possible heuristic: present examples with more error more often – but use with caution! Sometimes the hard examples are hard because:
 - a) there is an error in the training set – a pattern is incorrectly labeled!
 - b) more has to be learned **about the easy examples before the hard ones can be tackled (what does this mean?)** but, this approach can improve performance on infrequent examples, e.g., /z/ in phoneme recognition.

Tricks of the Trade

Changes to input handling:

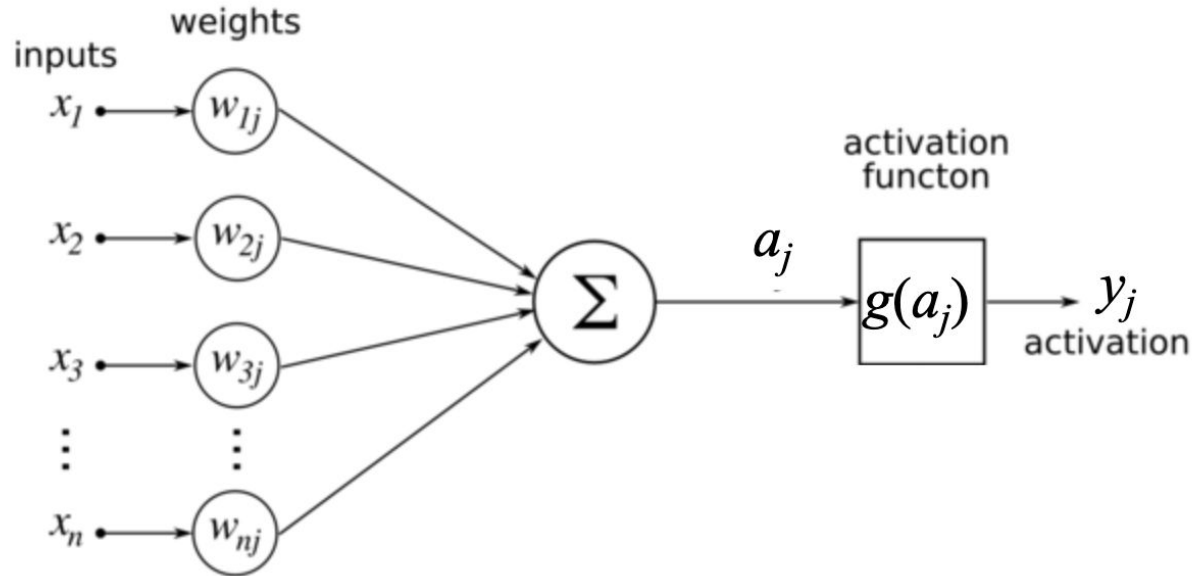
- Stochastic gradient descent versus batch learning
- Shuffling the examples
- **Perform PCA of the inputs**

Changes to architecture/model properties:

- Change the sigmoid
- Initialize the weights
- Momentum
- Adaptive learning rates

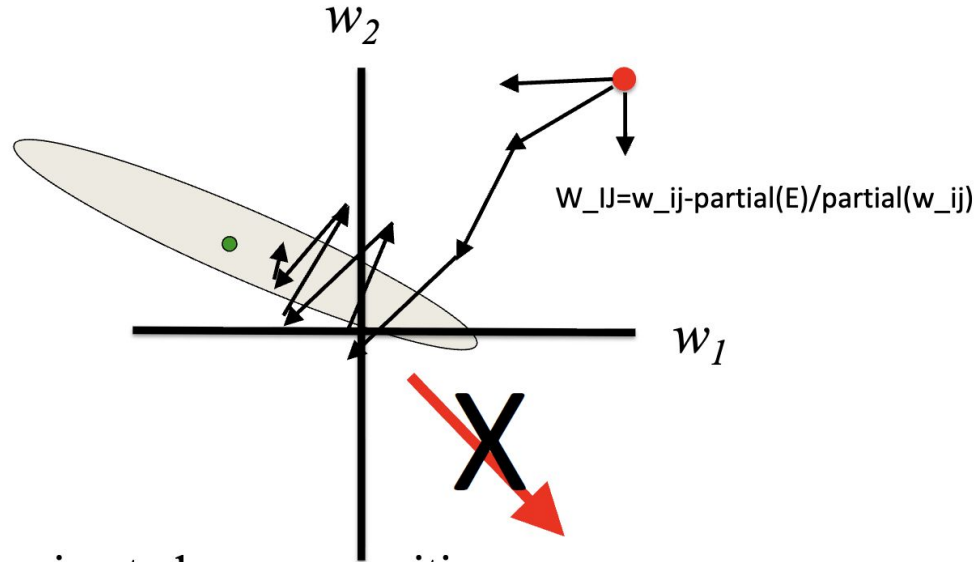
What's wrong with all positive inputs?

What happens to the **weight changes** for all of the incoming weights to one hidden unit, call it j ? Here, all the x_i 's are positive:



Weight changes all positive or negative: where can the weights go?

(See previous slide: this is for *one unit's incoming weights*)



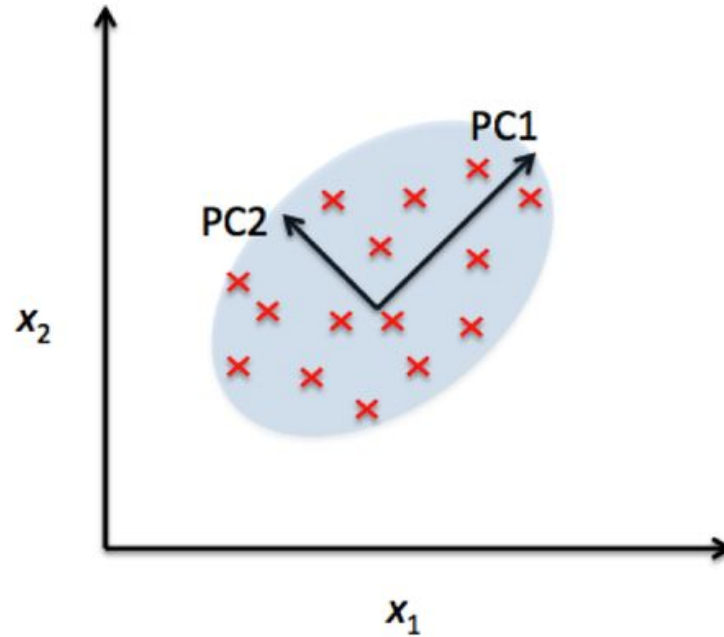
- Only changing to be more positive or more negative...never one positive and one negative.

What's wrong with having all inputs correlated?

- What patterns is your model learning?

PCA helps to decorrelate inputs

(And also has some of the same affordances as z scoring like shifting the mean to 0)



Tricks of the Trade

Changes to input handling:

- Stochastic gradient descent versus batch learning
- Shuffling the examples
- Perform PCA of the inputs

Changes to architecture/model properties:

- **Change the sigmoid**
- Initialize the weights
- Momentum
- Adaptive learning rates

Ok, we've done all these good things...

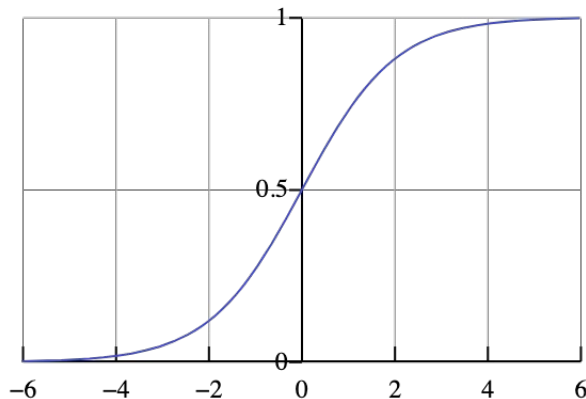
And now...we put these nicely formatted input into the logistic sigmoid????

- What will the activations be?
- Then the next layer up gets all...(negative? positive?)

Ok, we've done all these good things...

And now...we put these nicely formatted input into the logistic sigmoid?

- What will the activations be?
- Then the next layer up gets all positive input again!



We need a different activation function.

So, change the sigmoid to have the same properties as before - because these are inputs to the next layer.

- Standard sigmoid makes all inputs to the next layer positive - BAD.
- A “bipolar” sigmoid - one symmetric around 0 - is better.

≡ Sigmoid function

[Article](#) [Talk](#)

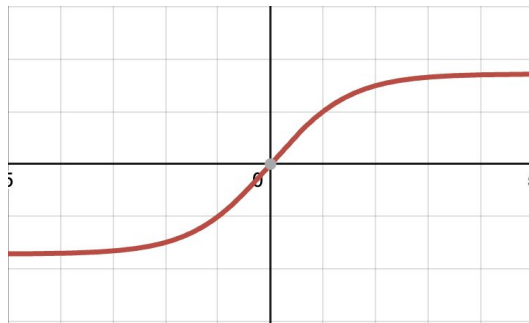
From Wikipedia, the free encyclopedia

A **sigmoid function** is any [mathematical function](#) whose [graph](#) has a characteristic S-shaped or **sigmoid curve**.

Recommended sigmoid: $f(x) = 1.7159 * \tanh(0.667x)$.

Why? If x has unit variance, then the output will have unit variance – the output is “formatted” for the next layer up.

- Empirical note—one helpful property: effective gain is 1 over the linear range (outputs are the same as input)



Tricks of the Trade

Changes to input handling:

- Stochastic gradient descent versus batch learning
- Shuffling the examples
- Perform PCA of the inputs

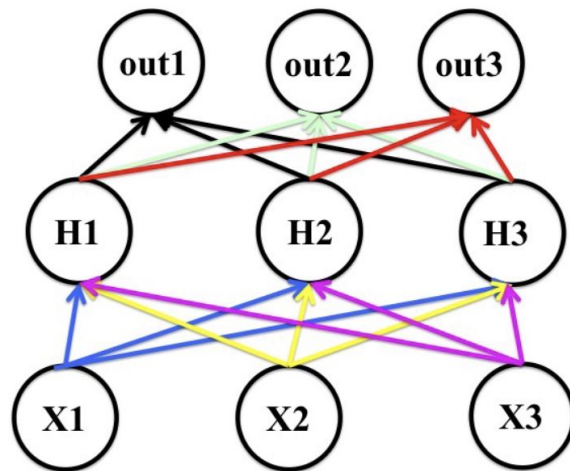
Changes to architecture/model properties:

- Change the sigmoid
- **Initialize the weights**
- Momentum
- Adaptive learning rates

Weight initialization

- Why can't the weights start at zero?
 - The δ 's of the hidden units will all be the same...
 - Inputs will be different...
 - So outgoing weights from each input will be the same...and the hidden units will all compute the same feature!
- Instead, we randomize weights. Many strategies for this!

$$w_{ij} = w_{ij} + \alpha \delta_j x_i$$



So, we start with *random* weights, which breaks this symmetry

Tricks of the Trade

Changes to input handling:

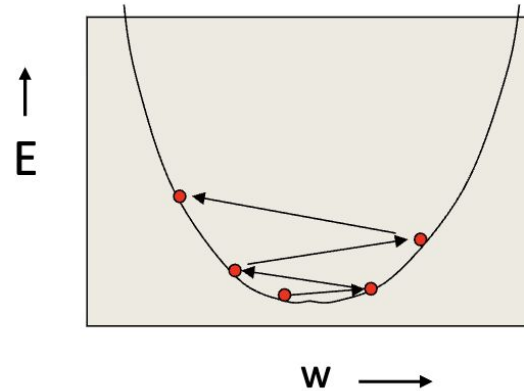
- Stochastic gradient descent versus batch learning
- Shuffling the examples
- Perform PCA of the inputs

Changes to architecture/model properties:

- Change the sigmoid
- Initialize the weights
- **Momentum**
- Adaptive learning rates

Why momentum? How the learning goes wrong

- If the learning rate is big, the weights sash to and fro across the ravine.
 - If the learning rate is too big, this oscillation diverges.
- What we would like to achieve:
 - Move quickly in directions with small but consistent gradients.
 - Move slowly in directions with big but inconsistent gradients.

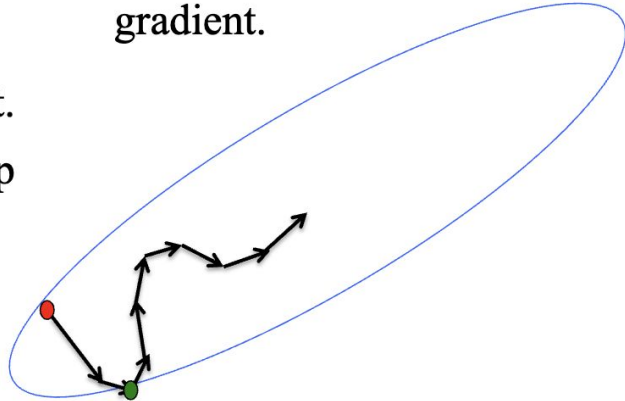


The intuition behind the momentum method

Imagine a ball on the error surface. The location of the ball in the horizontal plane represents the weight vector.

- The ball starts off by following the gradient, but once it has velocity, it no longer does steepest descent.
- Its momentum makes it keep going in the previous direction.

- Momentum damps oscillations in directions of high curvature by combining gradients with opposite signs.
- It builds up speed in directions with a gentle but consistent gradient.

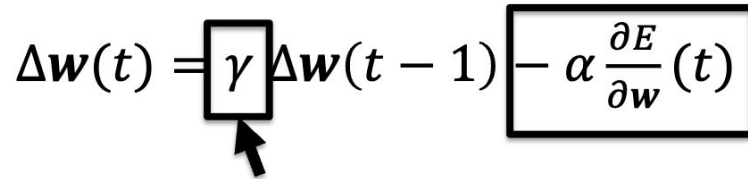


Why do we want smooth?

- Helps models bypass flat areas, reduce wild zig-zag movements, and move smoothly
- Methods like Stochastic Gradient Descent look only at the immediate slope. On a bumpy surface, they often bounce back and forth or get stuck in temporary low points (valleys)--momentum helps us look at things a bit more in aggregate before making a move

The equation of the momentum method

The effect of the gradient at time t is to increment the previous average.

$$\Delta \mathbf{w}(t) = \boxed{\gamma} \Delta \mathbf{w}(t-1) - \boxed{\alpha \frac{\partial E}{\partial \mathbf{w}}(t)}$$


The average also decays by γ which is slightly less than 1.

Thus, the weight change is the sum of the previous weight change and the new gradient.

The gamma tells us how much “momentum” to give the immediately preceding weight.

Tricks of the Trade

Changes to input handling:

- Stochastic gradient descent versus batch learning
- Shuffling the examples
- Perform PCA of the inputs

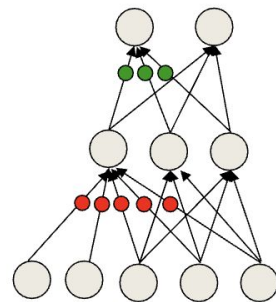
Changes to architecture/model properties:

- Change the sigmoid
- Initialize the weights
- Momentum
- **Adaptive learning rates**

The intuition behind separate adaptive learning rates

In a multilayer net, the appropriate learning rates can vary widely between weights:

- The magnitudes of the gradients are often very different for different layers, especially if the initial weights are small.
- The fan-in of a unit determines the size of the “overshoot” effects caused by simultaneously changing many of the incoming weights of a unit to correct the same error.



Gradients can get very small in the early layers of very deep nets.

The fan-in often varies widely between layers.

One way to determine the individual learning rates

- Start with a local gain of 1 for every weight ($g_{ij}=1$)
- Increase the local gain if the gradient for that weight does not change sign.
- Decrease the local gain if the gradient for that weight is changing sign

$$\Delta w_{ij} = -\varepsilon g_{ij} \frac{\partial E}{\partial w_{ij}}$$

$$\text{if } \left(\frac{\partial E}{\partial w_{ij}}(t) \frac{\partial E}{\partial w_{ij}}(t-1) \right) > 0$$

$$\text{then } g_{ij}(t) = g_{ij}(t-1) + .05$$

$$\text{else } g_{ij}(t) = g_{ij}(t-1) * .95$$

Questions?

Break!

See you in 5 mins.

Anonymous course feedback

- <https://forms.gle/JbPKVFQ6eU39q5Ym6>

End of where we reached in class